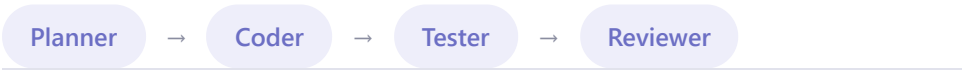


PROJECT DOCUMENTATION

Autonomous Coding Agent Swarm

A multi-agent orchestration system that plans, writes, tests, and reviews a real code change end-to-end — with every decision logged and visible.



BACKEND

Django 5 · DRF · Celery · Redis

FRONTEND

React 18 · Vite · React Router

LLM PROVIDER

Groq · 4 keys, one per agent role

ISOLATION

Docker (sandboxed, no network)

AUTH

Google / GitHub OAuth · email+password · JWT

1. What This Project Does

Agent Swarm automates a coding task the way a small engineering team would divide it up — except every "team member" is a specialized agent, and the whole exchange is logged. You sign in with a private account, then give it a plain-English task description and the URL of a public GitHub repository. From there, four cooperating agents take over:

- **Planner** reads the task and the repo's file tree, and produces an ordered, concrete implementation plan.
- **Coder** follows that plan and rewrites the files it calls for, in a throwaway local clone of your repo (nothing is pushed back — the diff is shown to you to apply yourself).
- **Tester** runs your *real* test suite inside an isolated, network-disabled Docker container and reports pass/fail — no LLM involved, because testing should be deterministic.
- **Reviewer** gives the final diff a fast sanity check against the plan before the task is marked done.

If the tests fail, the Coder gets another shot — with the actual failure output as context — up to a bounded number of retries, instead of looping forever or silently giving up. A React dashboard polls the backend and renders all of this as a live timeline (the **Agent Trace**), plus a diff viewer for exactly what the Coder changed.

Every task, run, and diff belongs to exactly one account and is invisible to everyone else — this is now a genuine multi-tenant product, not a single-user local tool: sign in with Google, GitHub, or email/password, and a **Task History** sidebar (searchable, with favorite/rename/archive/delete) keeps every past run one click away, the same shape as ChatGPT's or Claude's conversation history.

Why this exists

This is a portfolio project built to demonstrate multi-agent orchestration end-to-end: a real Planner → Coder → Tester → Reviewer loop, backed by an async task pipeline, sandboxed execution, and observability by default — rather than a single LLM call dressed up as "agentic."

2. Tech Stack

Layer	Technology	Role
Backend / API	Django 5 + Django REST Framework	Task/AgentRun models, REST endpoints, free admin audit-trail UI
Async pipeline	Celery + Redis	Runs the Planner→Coder→Tester→Reviewer chain off the request thread; Redis is both the broker and result backend
Database	SQLite (dev)	Standard Django ORM only — no SQLite-only features — so it's a config change away from Postgres
Sandbox	Docker, via the <code>docker</code> Python SDK	One ephemeral, network-disabled container per Coder-write / Tester run
Planner LLM	Groq (Llama 3.1 8B Instant), one dedicated key	Repo-wide reasoning and task decomposition into a step plan
Coder LLM	Groq, two dedicated keys	Whole-file rewrites, given the plan + relevant file contents + prior test failures — rotates least-recently-used-first between its two keys, with a per-key cooldown on rate limits and failover (Section 3)
Tester	pytest / npm test (no LLM)	Deterministic pass/fail inside the sandbox
Reviewer LLM	Groq (Llama 3.1 8B Instant), one dedicated key	Fast, bounded "does this diff match the plan" check
Auth	django-allauth + dj-rest-auth + djangorestframework-simplejwt	Google/GitHub OAuth, email/password, JWT issuance — not hand-rolled
Frontend	React 18 + Vite + React Router	Task submission, live Agent Trace timeline, diff viewer, auth pages, task history
Diff rendering	react-diff-viewer-continued	Renders the Coder's before/after per file
Styling	Custom neumorphic CSS (light + dark)	Soft-UI theme driven entirely by CSS custom properties, mobile-first breakpoints

3. Why This Tech Stack

One LLM provider (Groq), four API keys

Every LLM-backed agent runs on Groq. This is a deliberate design choice, not an API bolted on at random — and it's a revision of an earlier one: the pipeline originally split work across Google Gemini (Planner/Coder) and Groq (Reviewer), until Gemini's free tier turned out to be capped per Google Cloud *project* rather than per key. See the constraints callout below.

- **Planner** → **Groq**, one dedicated key. Reads repo-wide context — the file tree and task description — and returns an ordered step plan.
- **Coder** → **Groq, two dedicated keys**. The Coder is the highest-*volume* agent by far (one call per plan, plus one more per retry attempt) — so unlike the other three agents it isn't pinned to a single key. See the algorithm below.
- **Tester** → **no LLM at all**, and so no key. Test results should be deterministic and reproducible. Programmatically parsing a real `pytest` exit code and output is strictly more trustworthy than asking a model to "judge" whether tests passed.
- **Reviewer** → **Groq**, one dedicated key. A bounded, final "does this look reasonable" check needs to be fast and cheap, not clever. Groq's low latency makes it a snappy last step rather than a bottleneck.

All provider calls go through one abstraction, `agents/llm_client.py`'s `call_llm(provider, messages, api_key)`, so swapping which provider or key an agent uses is a one-line change — not a rewrite. The `provider` argument is kept even though only Groq is wired up today: `AgentRun.provider` records it per run, and adding a second provider back means adding one `_call_*` function plus one dispatch entry, not touching four agent modules. `api_key` is always passed in explicitly by the caller; the client module has no notion of a single global key. It also centralizes the free-tier handling every agent needs: exponential backoff with jitter on 429s, capped at 3 retries, then surfaced as a distinct `LLMRateLimitError` so callers can tell "this key is out of quota" apart from "this request was bad."

Four keys, one per agent role

Each agent *role* gets its own dedicated key rather than one shared Groq key across the whole pipeline, so a role that burns through its quota can't starve the others:

Role	Env var	Keys
Planner	<code>GROQ_API_KEY_PLANNER</code>	1
Coder (slot A)	<code>GROQ_API_KEY_CODER_A</code>	2
Coder (slot B)	<code>GROQ_API_KEY_CODER_B</code>	
Reviewer	<code>GROQ_API_KEY_REVIEWER</code>	1
Tester	—	0 (no LLM)

The Coder's switching algorithm: least-recently-used + per-key cooldown + failover

Implemented in `agents/services/key_pool.py`, which the Coder consults on every call:

1 Least-recently-used first.

Whichever of the two keys was used longest ago serves the next call. In the steady state that's plain alternation (A, B, A, B...), halving the request pressure on either key across a multi-retry run instead of hammering one key `MAX_TEST_RETRIES` times in a row.

2 Cooldown on rate limits.

A key that returns a 429 is benched for a period that *doubles* per consecutive rate limit (60s → 120s → 240s, capped at 480s), so the next call skips straight to its sibling rather than paying `call_llm`'s full backoff cycle against a key already known to be out of quota. A success clears the penalty. A non-429 failure — bad key, malformed request, transient 5xx — gets a much shorter 15s bench instead: enough to prefer the sibling on the next call, without writing off what might have been a single blip.

3 Failover, and never a hard stop.

If the chosen key's call fails for any reason, the Coder immediately tries the other before giving up that attempt. Benched keys are still offered as a last resort, ordered by whose cooldown expires soonest — so even with both keys throttled, a call attempts the one closest to recovery rather than failing outright.

Why LRU rather than a plain alternate-by-attempt-number counter. A fixed counter keeps sending every even attempt to key A even when A is known to be rate-limited, wasting a full backoff cycle before failing over — every single time. LRU plus cooldown skips the dead key outright, and it self-corrects: the key that got skipped is the stalest one once its cooldown expires, so it's picked first and the load re-balances on its own instead of staying permanently lopsided. Recency is tracked with a pool-wide counter rather than a timestamp, because `time.monotonic()`'s ~15ms resolution on Windows makes two nearby calls read as equally recent — which ties the comparison and silently collapses the rotation onto one key.

Pool state is per-process and in-memory, which under `celery -P solo` (what this project documents on Windows) is exactly one pool shared by every pipeline run. Under a multi-process worker each process keeps its own view — still correct, just less well-informed; moving the pool to Redis is the upgrade path, and nothing outside `key_pool.py` would change. `AgentRun.output["key_slot"]` records which key actually served a run — the Agent Trace surfaces it as *via groq · key A* — and `["keys_tried"]` keeps the full attempt list regardless of outcome, so a silent failover stays visible after the fact instead of looking like a clean first-try success.

Surviving a model that mangles its own output

The most common way this pipeline fails in practice is not a bad plan or bad code — it's the model breaking the output contract. Two layers handle that:

- `parse_strict_json` **tolerates chatter, not corruption** (`services/common.py`). Every prompt says "strict JSON only, no prose, no fences", and an 8B model mostly complies; when it doesn't, the response is usually still salvageable. It locates the first JSON value and uses `raw_decode`, which stops

at the end of that value rather than demanding the whole string be consumed — handling leading prose, ````json` fences, and the one that actually bit: *anything after* the JSON, where plain `json.loads` throws away a perfectly good response with `Extra data: line 2 column 1`. Genuinely truncated or corrupt output is still rejected.

- **The Coder re-asks on unusable output** (`MAX_MALFORMED_RETRIES` , default 2). A response that stops mid-string is transient rather than deterministic — the same prompt against the same repo produced clean JSON on one call and a response truncated at 2051 characters on the next, with Groq reporting `finish_reason="stop"` both times, so there is nothing to detect up front. Without a re-ask a single such response aborts the whole run, because `tasks.py` only loops the Coder back around when the *Tester* rejects the code — never when the Coder produced no usable code at all. Each re-ask carries a short corrective note (deliberately *not* the bad output: re-sending multiple KB of half-escaped file content would blow the token ceiling and turn a recoverable blip into a hard 413) and naturally lands on the other API key, since the pool hands out least-recently-used first.

The Coder also de-duplicates repeated paths in the `files` list, keeping the last entry. A model that emits `app.py` twice would otherwise have that file written twice, with the second `file_diffs` entry capturing the first rewrite as its "before" — a phantom diff between two generated versions rather than against the real original.

Known free-tier constraints (found in practice, not guessed)

Both of these surfaced with live keys, and between them they shaped the whole provider setup:

- **Groq's llama-3.1-8b-instant free tier caps at ~6000 tokens/minute**, combined prompt + completion, per request — the single biggest constraint on this pipeline. Requesting `max_output_tokens=16384` fails every single time with a 413 ("Request too large ... Limit 6000, Requested 18269"): the output-token ask alone exceeds the ceiling before the prompt is even counted. So every agent reserves only what its output actually needs, leaving the rest of the budget for its prompt — Coder `3000` (`_CODER_MAX_OUTPUT_TOKENS`), Planner `1500` (`MAX_PLAN_OUTPUT_TOKENS` ; its prompt carries the repo file tree), Reviewer `1024` (`MAX_REVIEW_OUTPUT_TOKENS` ; its prompt carries 6000 chars of diff). A response that still gets truncated comes back as `finish_reason=length` , which `llm_client.py` raises as an explicit token-budget error rather than letting it surface downstream as a baffling JSON parse failure.
- **Why Gemini was dropped: its free tier is limited per Google Cloud *project*, not per API key** — confirmed via the actual 429 payload:
`quotaId: GenerateRequestsPerDayPerProjectPerModel-FreeTier` , `limit: 20` requests/day. Two API keys issued from the same Google account under the same project share that one 20/day pool — they do not double it, and AI Studio's default "quick create" flow reuses the same project every time. 20 requests/day doesn't survive normal development on a pipeline that can make four Coder calls in a single run. Groq's per-key limits, by contrast, are genuinely independent per key — which is what makes the two-key Coder rotation worth anything at all.

This combination gets a lot for free that a leaner stack would have to build by hand. The Django admin doubles as a zero-extra-frontend audit trail for every `Task` and `AgentRun` — genuinely useful for debugging a multi-step agent pipeline, not just a scaffold artifact. Celery is the standard way to move a multi-minute, multi-API-call pipeline off the HTTP request/response cycle, and Redis serving as both the broker and result backend keeps the infrastructure to a single extra service in dev.

Docker sandbox, not a subprocess

Executing LLM-generated code is the one place in this project where isolation is non-negotiable. Every Coder-written file is exercised inside a throwaway container with `network_disabled=True`, a memory cap, and an enforced timeout — the generated code cannot reach the network, exhaust the host, or hang the pipeline. A plain subprocess call would not give any of those guarantees.

Whole-file rewrites, not LLM-generated diffs

The Coder agent outputs strict JSON — `{"files": [{"path", "content"}]}` — with the complete new content of each file it touches, rather than a unified diff. LLM-generated unified diffs are fragile to parse: a single off-by-one context line breaks `git apply` outright. Whole-file rewrites are trivial to validate and apply safely (including a path-traversal check before anything touches disk). A real `git diff` is then computed *after* the files are written, so the Reviewer prompt and the frontend's diff viewer still see a proper diff — the model just never has to produce one directly.

React + Vite, polling first

The dashboard's job is to make an asynchronous, multi-minute pipeline legible in real time. Plain polling (every 2–3 seconds) gets that job done with a fraction of the complexity of a WebSocket layer, and it's what actually shipped for v1 — Django Channels remains a clearly-scoped stretch goal rather than a requirement the demo depends on.

django-allauth + dj-rest-auth, not hand-rolled OAuth

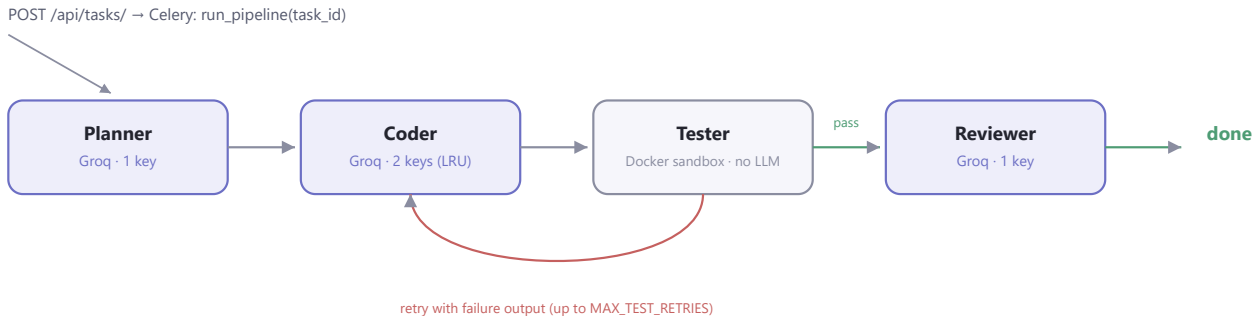
OAuth's code-exchange flow, CSRF/state handling, and per-provider quirks (Google vs. GitHub have different token and profile-fetch shapes) are exactly the kind of security-critical, easy-to-get- subtly-wrong code worth not writing from scratch. `django-allauth` already implements all of it, tested across a huge number of real deployments; `dj-rest-auth` wraps it in a JSON API instead of allauth's default server-rendered pages, which is what an SPA needs.

Both JWTs as httpOnly cookies

The access *and* refresh tokens are delivered as `httpOnly`, `SameSite=Lax` cookies — never handed to page JavaScript, never touching `localStorage`. This closes off the most common JWT footgun (an XSS payload reading a token out of storage and exfiltrating it) at essentially no extra cost, since `dj-rest-auth` supports this mode natively. The frontend never sees or manages a token at all; it just sends requests with `withCredentials: true` and lets the browser do the rest.

4. How It Works

Pipeline flow



Step by step

- 1 Submit**
 - A task (description + public GitHub repo URL) is created via `POST /api/tasks/` and enqueued as a Celery job; the worker shallow-clones the repo into a throwaway temp directory.
- 2 Plan**
 - The Planner reads the task description and a trimmed repo file tree, and returns strict JSON: an ordered list of concrete implementation steps.
- 3 Code**
 - The Coder selects the files the plan actually names (or falls back to a size-capped sweep), sends their contents plus the plan to Groq (on whichever of its two keys the pool picks), and gets back whole-file rewrites. Those are written straight into the repo; a real `git diff` is computed immediately after.
- 4 Test**
 - The modified repo is bind-mounted into a fresh, network-disabled Docker container and its real test suite runs (`pytest` by default). Exit code and output are parsed — no model involved.
- 5 Retry if needed**
 - On failure, the Coder runs again with the actual test output as context, up to `MAX_TEST_RETRIES` (default 3) attempts, before the task is marked `failed`.
- 6 Review**
 - Once tests pass, the Reviewer checks the final diff against the plan and returns `{"approved", "concerns"}`.
- 7 Done**
 - The task is marked `done`. Every stage's full input/output — including the raw LLM response — is already sitting in the database.

Home

Dark

shouryashobhit1

New Task

Description

e.g. Add input validation to the /signup endpoint

GitHub repo URL

https://github.com/owner/repo

Run Agent Swarm

Task History

Search tasks...

Recent Favorites Archived

DONE

Add input validation to the /signup endpoint in app....

https://github.com/shob0902/demo_repo_broken

☆ ✎ 🔄 📄

FAILED

Add input validation to the /signup endpoint in app....

https://github.com/shob0902/demo_repo_broken

☆ ✎ 🔄 📄

Agent Trace — Add input validation to the /signup endpoint in app.py: reje...

Planner via groq 436 ms SUCCESS

Coder via groq · key B 983 ms SUCCESS

Tester 1342 ms SUCCESS

Coder attempt 2 via groq · key A 578 ms SUCCESS

Tester attempt 2 1032 ms SUCCESS

Reviewer via groq 218 ms SUCCESS

Diff

app.py

before

1 1 import email

2 - from email import validate_email_address

3 2 from flask import Flask, jsonify, request

3 + import re

4 4

5 5 app = Flask(__name__)

6 6

Expand 5 lines ...

12 12 email = data.get("email", "")

13 13 password = data.get("password", "")

14 14

15 - if not validate_email_address(email):

15 + if not re.match(r"^[a-zA-Z0-9_%-]+@[a-zA-Z0-9-]+\.[a-zA-Z]{2,}\$", email):

16 16 return jsonify({'error': 'Invalid email'}), 400

17 17

18 18 if not password:

Expand 7 lines ...

A complete run against the deliberately-broken demo repo — every step above, actually happening. The **retry loop fires**: the Coder's first rewrite doesn't satisfy the repo's real test suite, the Tester's pytest output is fed back as context, and *Coder attempt 2* gets it right — after which the Reviewer approves and the task lands on **done**. The **two-key rotation is visible too**: attempt 1 was served by *key B* and the retry by *key A*, exactly the least-recently-used ordering described in Section 3.

The diff below the trace is a genuine **git diff** computed from the clone, not model-generated — here the Coder replaced a hallucinated **email** -module import with a stdlib **re** pattern.

Observability by default

Every agent invocation creates an **AgentRun** row *before* it calls the LLM or the sandbox (status **pending**), and updates it afterward — success or exception — so a crashed agent is always visible in the trace, never

silently missing. Every stage's raw LLM response is persisted to `AgentRun.output`, so a failure can be diagnosed from the database without re-calling the API. All of this is visible two ways: the React dashboard's live Agent Trace, and the Django admin (`/admin/`), which needed zero extra frontend work to become a full audit log.

Data model

Model	Key fields
User accounts app	<code>email</code> (unique, login identity), <code>name</code> , <code>avatar_url</code> , <code>signup_provider</code> , <code>role</code> , <code>date_joined</code> — no <code>username</code> field at all
Task	<code>user</code> (FK, ownership), <code>title</code> , <code>description</code> , <code>github_url</code> , <code>status</code> (pending → planning → coding → testing → review → done/failed), <code>is_favorite</code> , <code>is_archived</code>
AgentRun	<code>task</code> (FK), <code>agent_type</code> , <code>provider</code> , <code>input_context</code> , <code>output</code> , <code>status</code> , <code>retry_count</code> , <code>duration_ms</code>

`AgentRun` deliberately has no `user_id` of its own — ownership is enforced transitively through `AgentRun.task.user`, its existing FK relationship. Duplicating the owner onto every child table would just be redundant, driftable state; every query that needs to check "is this mine" already goes through the one `Task.user` column.

5. Authentication & Multi-Tenancy

Every account is private: signing in is required before any task can be created, and every API endpoint only ever returns data owned by the caller. This section covers how accounts work and how isolation is enforced.

Signing in

- **Email + password** — works today, no external setup required. Signup logs you in immediately (no email-verification step configured for this project).
- **Google / GitHub OAuth** — fully implemented end-to-end, but each provider needs a real Client ID/Secret registered in that provider's own developer console before its button goes live (`backend/.env.example` and `frontend/.env.example` document exactly what to create and where). Until then, the buttons render disabled with a tooltip rather than pretending to work.
- **One account per email, regardless of provider** — if the same email signs up via email/password and later clicks "Continue with Google," a custom `SocialAccountAdapter` connects the new login to the existing account instead of erroring or creating a duplicate.

Session mechanics

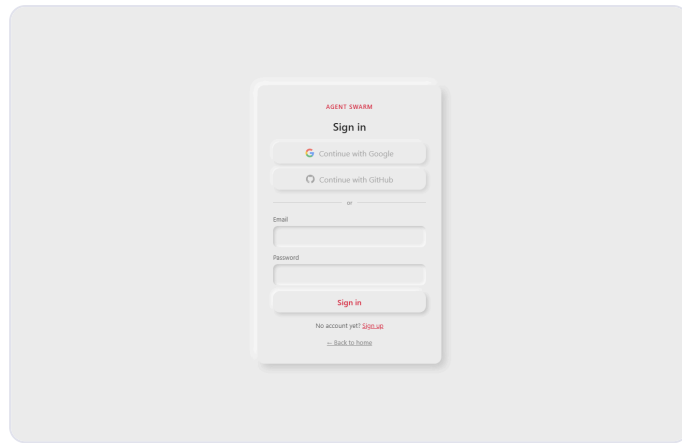
The backend issues a short-lived JWT access token and a longer-lived refresh token, both as `httpOnly` cookies (see Section 3 for why). The frontend's axios client sends `withCredentials: true` on every request; on a `401` it silently calls the refresh endpoint once and retries the original request before giving up and redirecting to `/login`. Logging out blacklists the refresh token server-side, not just clears the cookie, so a stolen-but-unused refresh token can't be replayed after logout.

Data isolation

Every `Task` list endpoint filters to `request.user`'s own rows only — another user's tasks are never even considered, let alone returned. Reaching for a specific task/run by id that belongs to someone else returns **403 Forbidden**, not a 404 — Django REST Framework's default object-permission behavior once an `IsOwner` permission class is wired in, so no special-casing was needed to meet that requirement. All of this sits behind a global default of `IsAuthenticated`, so a new endpoint is private by default and has to opt out, not the other way around.

Task History

The sidebar (previously a flat task list) now supports free-text search over title/description, three filters (Recent / Favorites / Archived), inline rename, a favorite/pin toggle, archive (reversible, hides from "Recent"), and delete (permanent, confirmed). All of it is scoped to the signed-in user the same way task creation is.



Sign-in page — OAuth buttons render disabled until real provider credentials are configured; email/password works immediately.

6. How to Start and Run It

Prerequisites

- Python 3.11+, Node 18+, Git
- **Docker Desktop**, running — required for the sandbox and for Redis via compose
- **Four free API keys**, all from [Groq](#) — one Groq account can issue multiple keys, so this doesn't need four separate accounts. See "Four keys, one per agent role" in Section 3.

Setup

1 Start Redis

```
docker compose up -d
```

2 Build the sandbox image (once)

— `python:3.11-slim` alone has no test runner, and the sandbox has no network access at run time to install one.

```
cd backend
docker build -t agent-swarm-sandbox:latest -f docker/sandbox.Dockerfile .
```

3 Backend — install & migrate

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env          REM then fill in the four GROQ_API_KEY_* vars (Section 3)
                                REM (GOOGLE/GITHUB_CLIENT_ID+SECRET optional -- see Section 5)

python manage.py migrate
python manage.py createsuperuser REM optional: gives you /admin/ access
python manage.py test_llm        REM optional: tests each of the four API keys individually
```

4 Backend — run the API server

(terminal 1)

```
python manage.py runserver
```

5 Backend — run the Celery worker

(terminal 2 — Windows needs `-P solo`)

```
celery -A orchestrator worker --loglevel=info -P solo
```

6 Frontend

(terminal 3)

```
cd frontend
npm install
copy .env.example .env    REM optional: VITE_GOOGLE_CLIENT_ID / VITE_GITHUB_CLIENT_ID
npm run dev
```

Open the URL Vite prints — by default `http://localhost:5173` .

Try it — the built-in demo task

Sign up (email/password is enough) — you'll land on the dashboard. A ready-made target repo ships in `demo_repo/` : a tiny Flask app whose `/signup` endpoint has no validation, with tests that fail against it. Push that folder to a public GitHub repo of your own (tasks now take a `github_url` , not a local path — Section 4's *Coder* note explains why), then submit:

Description:

Add input validation to the `/signup` endpoint: reject requests with an invalid email format or a missing password with a 400 response.

GitHub repo URL:

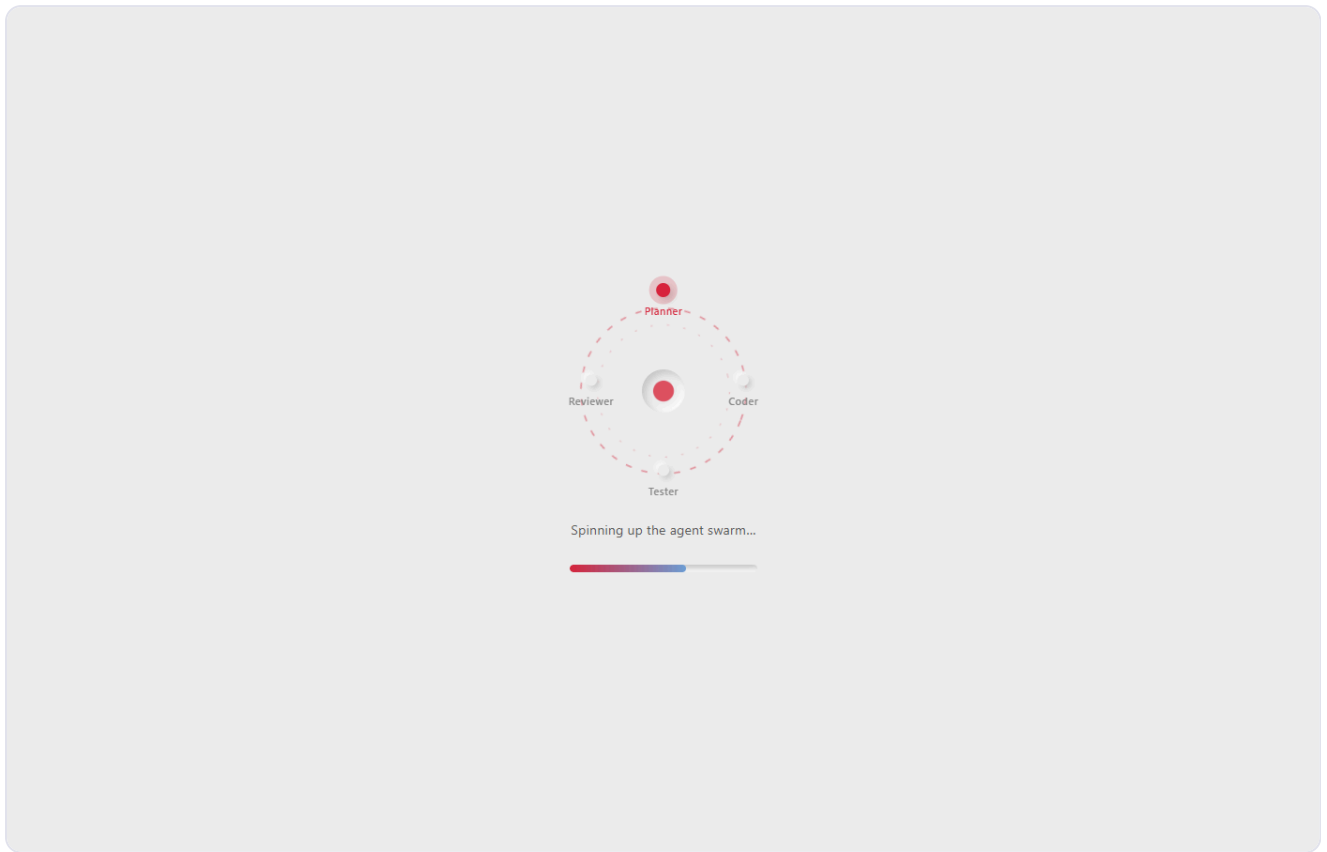
`https://github.com/<you>/demo_repo`

Watch the Agent Trace go *Planner* → *Coder* → *Tester* → *Reviewer* and land on `done` . It'll show up in the Task History sidebar afterward — searchable, favorable, renameable — right where you'd expect it in a ChatGPT/Claude-style history list.

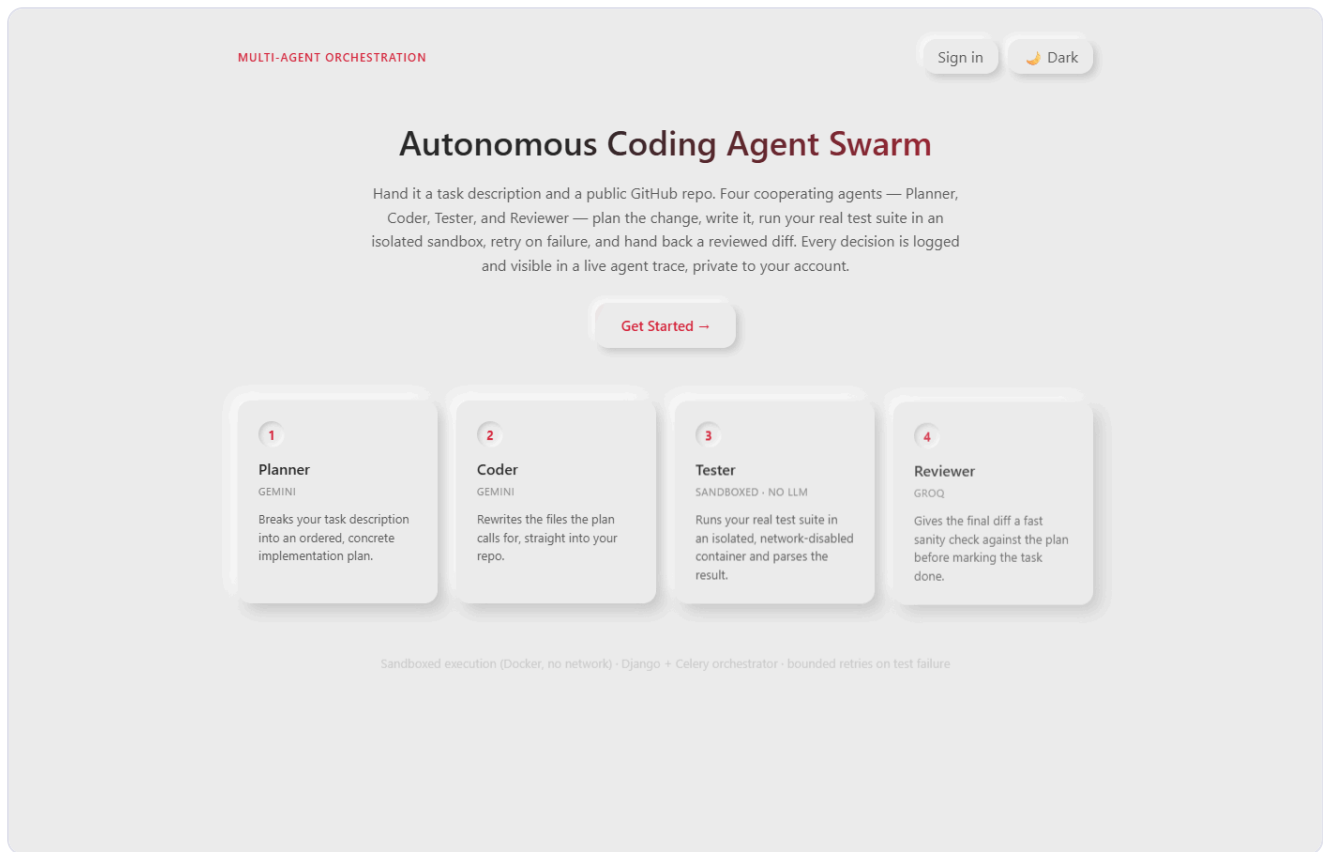
Troubleshooting notes from real setup runs

- **'docker' is not recognized in PowerShell** — a per-user Docker Desktop install doesn't always add itself to PATH. Use the full path once (`C:\Users\<you>\AppData\Local\Programs\DockerDesktop\resources\bin\docker.exe`), or add that folder to your user `PATH` permanently and reopen the terminal.
- **"Failed to create task" with no POST logged on the backend** — usually a leftover `npm run dev` process holding port 5173, pushing your actual tab onto 5174/5175, which the backend's `CORS_ALLOWED_ORIGINS` doesn't trust. Close stray dev-server terminals, or widen the allow-list in `backend/.env` .
- **Backend picks up .env only at startup** — restart `runserver` after editing it.
- **"Continue with Google/GitHub" stays greyed out** — expected until real `CLIENT_ID` / `CLIENT_SECRET` values are set in both `backend/.env` and `frontend/.env` (Section 5). Email/password isn't affected.

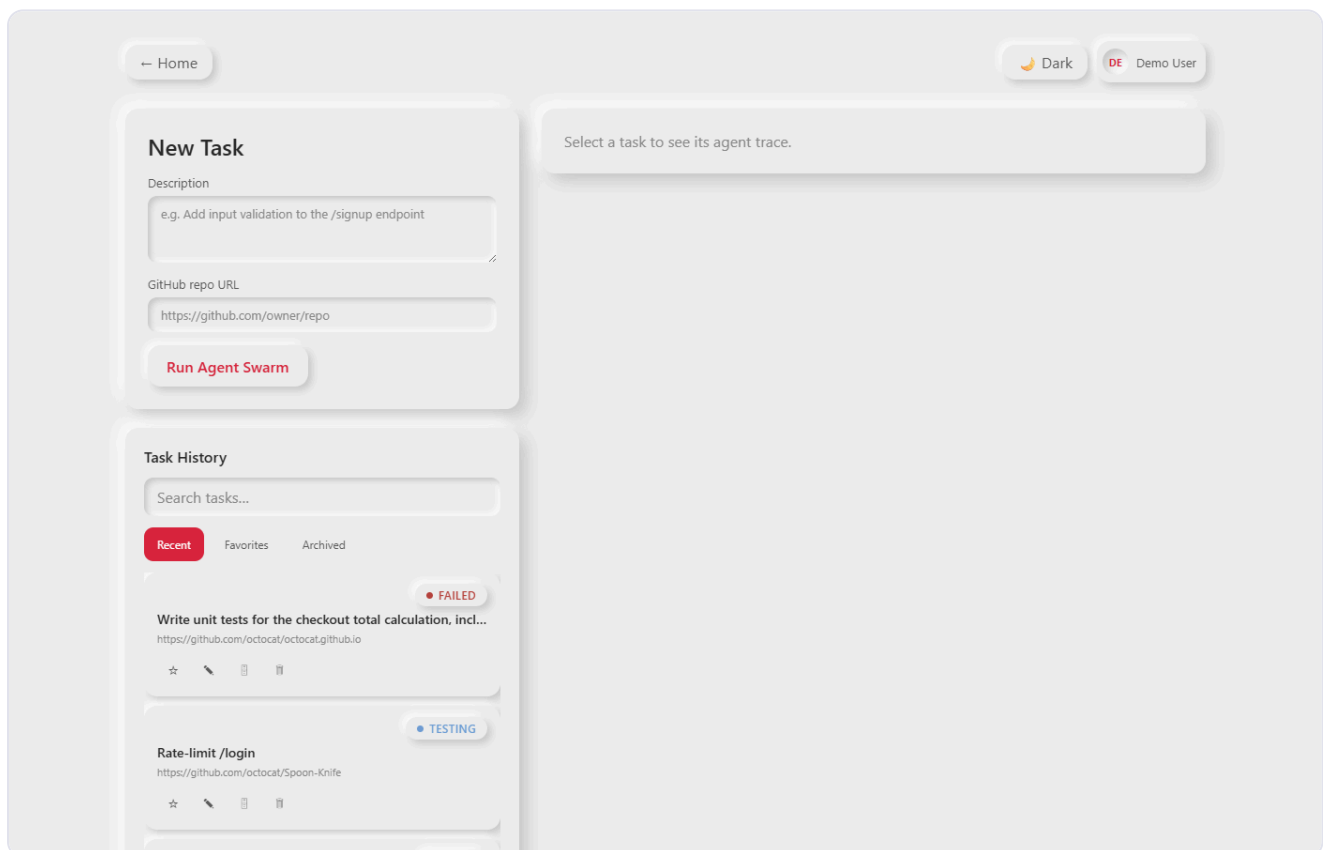
7. Screenshots



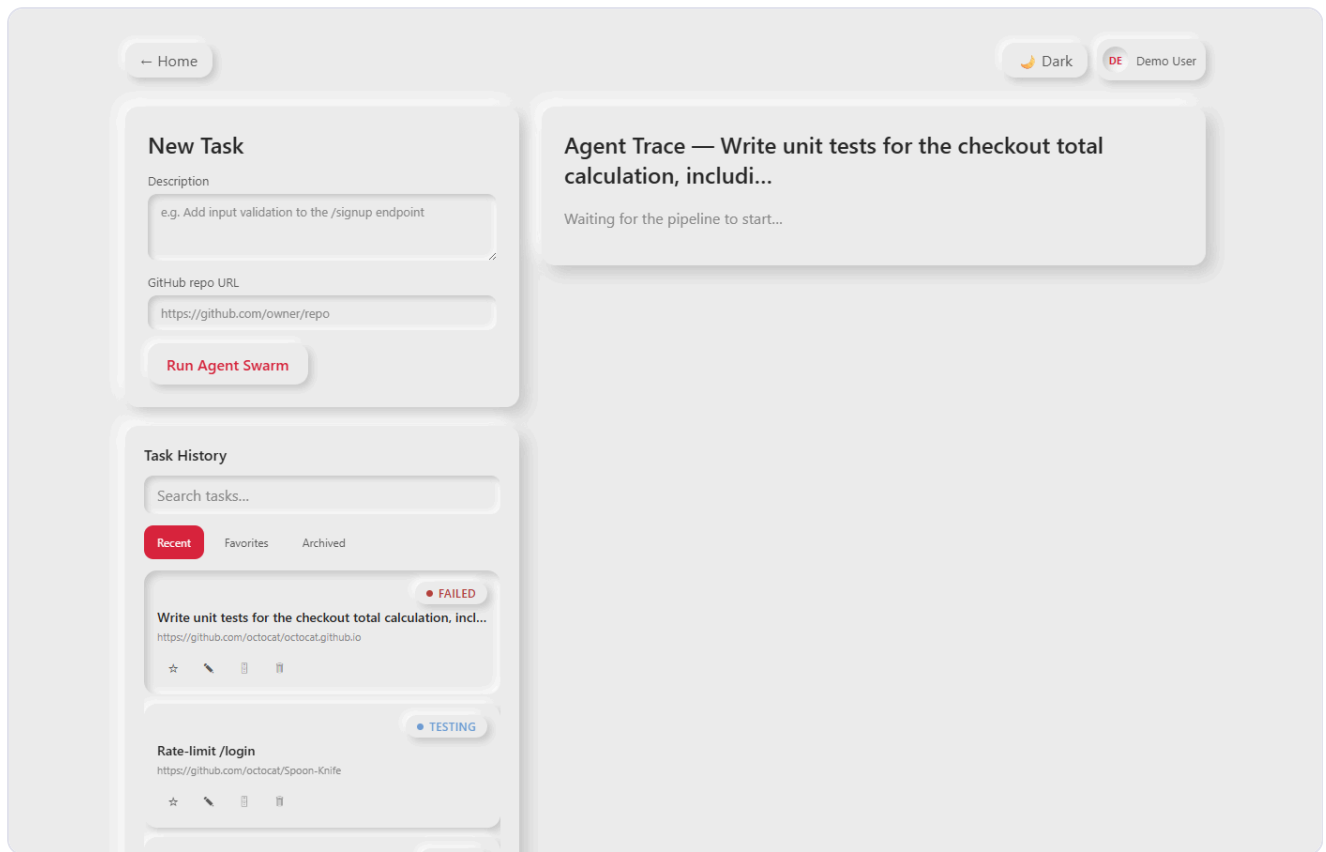
Splash screen shown once on load — four agent nodes cycling around a pulsing core before the landing page fades in.



Landing page — explains the pipeline (Planner → Coder → Tester → Reviewer) and links to sign-in, or straight to the dashboard once you're already signed in.



Dashboard — the New Task form now takes a GitHub URL, and the Task History sidebar (search, Recent/Favorites/Archived filters, per-task favorite/rename/archive/delete) replaces the old flat task list.



Selecting a task shows its live Agent Trace — unchanged in spirit from v1, now sitting alongside the account-scoped history and user menu.

8. Non-Goals (v1)

- No multi-repo support — one `github_url` per task
- Public GitHub repos only — no private-repo auth, no non-GitHub git hosts
- No real OAuth without setup — Google/GitHub sign-in needs a Client ID/Secret registered by whoever deploys this; ships fully wired but inert until then
- No email verification / password-reset delivery — no SMTP configured, so signup logs you in immediately rather than gating on a verification email
- No org/team accounts — one flat namespace of individual users; `role` exists on the User model as a future-facing hook, not an enforced permission tier today
- No LLM-based test generation — the Tester only runs a repo's existing tests
- No code execution outside the sandboxed container, ever